Technical Assessment: Software Engineering Intern

Time Limit: 3 Hours
Allowed Resources: Documentation, StackOverflow, and AI Assistants (ChatGPT, Claude, Gemini, etc.)
Submission Requirements: A GitHub repository link (or ZIP file) containing your source code, a sample output file, and a short `PROMPT_LOG.md` detailing how you used AI tools.

Objective

You will build a **Device Log Summarizer CLI tool**. This task evaluates your ability to write clean code, manipulate CSV data, handle basic errors, and effectively guide an AI assistant to build a helper visualization module.

Section 1: Coding Logic & Sanitization (30 – 45 Minutes)

Goal: Test fundamental data cleaning and edge-case handling without using AI.

The Problem: Clean and Normalize Sensor Readings

You are given a list of sensor values that may contain strings, `None` (nulls), or negative error flags.

Write a function in **Python** that takes a list of mixed inputs and returns a cleaned list of floats by applying these steps:

1. **Type Conversion & Filtering:** Convert elements to floats where possible. Drop any values that cannot be converted, are `None`, or are negative (< 0).
2. **Clipping:** If any valid number exceeds `500.0`, cap/limit it to `500.0`.
3. **Edge Case Handling:** Handle empty lists or lists with entirely invalid entries gracefully without throwing an unhandled exception.

Deliverable: Write the function along with 2-3 simple test cases.

Section 2: Core Data Processing Task (75 – 90 Minutes)

Goal: Test file reading, grouping, basic statistics, and JSON export.

You are provided with a small CSV file named `device_telemetry.csv` containing these columns:

- `device_id` (String, e.g., `DEV_A`, `DEV_B`)
- `status` (`ACTIVE`, `WARNING`, `ERROR`)
- `temperature` (Float)

Requirements:

1. **Data Ingestion:** Read the CSV file using Python (you can use standard `csv` library or `pandas`).
2. **Aggregation:** Group the data by `device_id` and compute:
    - Total number of records per device
    - Average (mean) temperature
    - Count of records where `status` is `ERROR`
3. **Filtering:** Filter out any devices that have an average temperature above `80.0`.
4. **Export:** Save the final aggregated results into a clean **JSON file** and print a summary table in the terminal.

Section 3: AI Prompting & Charting Challenge (45 – 60 Minutes)

Goal: Test your ability to prompt an LLM and fix minor bugs in its output.

1. Ask an AI assistant to write a Python script that loads your JSON output from Section 2 and generates a **bar chart** comparing the average temperature per device using `matplotlib`.
2. Review the AI's code, fix any layout or formatting bugs (like overlapping labels or missing titles), and run it to generate the chart.
3. **Deliverable (`PROMPT_LOG.md`):** Briefly document:
    - Your initial prompt to the AI.
    - One bug or formatting mistake the AI made in its first attempt.
    - The follow-up prompt you used to fix it.

Evaluation Criteria

- **Readability:** Clean variable names and logical code structure.
- **Robustness:** Does the code crash if the CSV file is missing or contains bad data?
- **AI Collaboration:** Evidence that you intelligently directed the AI rather than blindly copying flawed code.